

Информация

Докладчик

..... {column align=center}

:: {column width="70%"}

- Эспиноса Василита Кристина Микаела
- студентка
- Российский университет дружбы народов
- 1032224624@pfur.ru
- <https://github.com/crisespinosal>

::

:: {column width="30%"}

::

.....

Цель работы

Провести моделирование системы массового обслуживания (СМО).

Задание

- Реализовать модель $M|M|1$;
- Посчитать загрузку системы и вероятность потери пакетов;
- Построить график изменения размера очереди.

Выполнение лабораторной работы

$M|M|1$ — однолинейная СМО с накопителем бесконечной ёмкости. Поступающий поток заявок — пуассоновский с интенсивностью (λ). Времена обслуживания заявок — независимые в совокупности случайные величины, распределённые по экспоненциальному закону с параметром (μ).

Перейдём к реализации системы. Зададим параметры: ($\lambda = 30$), ($\mu = 33$), размер очереди — 100000, длительность моделирования — 100000.

Определим узлы сети и соединим их симплексным каналом с пропускной способностью 100 Кб/с, задержкой 0 мс и очередью типа DropTail, установив ограничение на её размер. В качестве источника трафика выберем UDP-агент, приёмником будет Null-агент. Также настроим мониторинг очереди. Процедура finish будет отвечать за закрытие файлов трассировки, а процедура sendpack — за генерацию пакетов по экспоненциальному распределению.

В данном сценарии дополнительно рассчитываются коэффициент загрузки системы и вероятность потери пакетов.

Выполнение лабораторной работы

Разработаем сценарий, реализующий модель согласно описанию в Xgraph график изменения ТСП-окна, график изменения длины очереди и средней длины очереди.

```
# создание объекта Simulator
set ns [new Simulator]
# открытие на запись файла out.tr для регистрации событий
set tf [open out.tr w]
$ns trace-all $tf
# задаём значения параметров системы
set lambda 30.0
set mu 33.0
# размер очереди для M|M|1 (для M|M|1|R: set qsize R)
set qsize 100000
# устанавливаем длительность эксперимента
set duration 1000.0
# задаём узлы и соединяем их симплексным соединением
# с полосой пропускания 100 Кб/с и задержкой 0 мс,
# очередью с обслуживанием типа DropTail
set n1 [$ns node]
set n2 [$ns node]

set link [$ns simplex-link $n1 $n2 100kb 0ms DropTail]
# наложение ограничения на размер очереди:
$ns queue-limit $n1 $n2 $qsize
# задаём распределения интервалов времени
# поступления пакетов и размера пакетов
set InterArrivalTime [new RandomVariable/Exponential]
$InterArrivalTime set avg_ [expr 1/$lambda]
set pktSize [new RandomVariable/Exponential]
$pktSize set avg_ [expr 100000.0/(8*$mu)]
# задаём агент UDP и присоединяем его к источнику,
# задаём размер пакета
set src [new Agent/UDP]
$src set packetSize_ 100000
$ns attach-agent $n1 $src
# задаём агент-приёмник и присоединяем его
set sink [new Agent/Null]
$ns attach-agent $n2 $sink
$ns connect $src $sink
# мониторинг очереди
set qmon [$ns monitor-queue $n1 $n2 [open qm.out w] 0.1]
$link queue-sample-timeout
# процедура finish закрывает файлы трассировки
proc finish {} {
    global ns tf
    $ns flush-trace
    close $tf
    exit 0
}
```

Выполнение лабораторной работы

```
# процедура случайного генерирования пакетов
proc sendpacket {} {
    global ns src InterArrivalTime pktSize
    set time [$ns now]
    $ns at [expr $time +[$InterArrivalTime value]] "sendpacket"
    set bytes [expr round ([$pktSize value])]
    $src send $bytes
}

# планировщик событий
$ns at 0.0001 "sendpacket"
$ns at $duration "finish"

# расчет загрузки системы и вероятности потери пакетов
set rho [expr $lambda/$mu]
set ploss [expr (1-$rho)*pow($rho,$qsize)/(1-pow($rho,($qsize+1)))]
puts "Теоретическая вероятность потери = $ploss"

set aveq [expr $rho*$rho/(1-$rho)]
puts "Теоретическая средняя длина очереди = $aveq"

# запуск модели
$ns run
```

Выполнение лабораторной работы

Запустив эту программу, получим значения загрузки системы и вероятности потери пакетов (рис. [-@fig:003]).

```
openmodelica@openmodelica-VirtualBox:~/work$ ns lab3.tcl
Теоретическая вероятность потери = 0.0
Теоретическая средняя длина очереди = 9.0909090909090864
```

Выполнение лабораторной работы

В каталоге проекта создадим отдельный файл, например `graph_plot`, с помощью команды `touch graph_plot`. Затем откроем его для редактирования и добавим код, следуя синтаксису GNUplot (см. рисунок [-@fig:004]).

```
#!/usr/bin/gnuplot -persist
# задаём текстовую кодировку,
# тип терминала, тип и размер шрифта

set encoding utf8
set term pdfcairo font "Helvetica,9"

# задаём выходной файл графика
set out 'qm.pdf'

# задаём название графика
set title "График средней длины очереди"

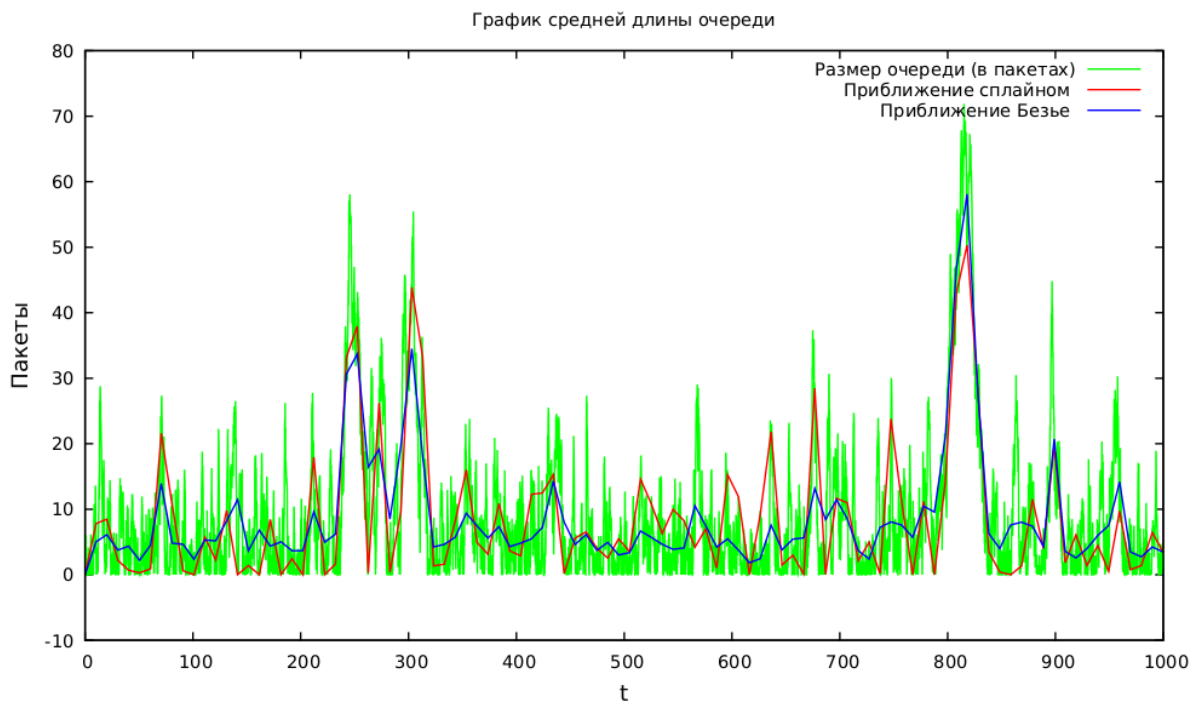
# подписи осей графика
set xlabel "t" font "Helvetica, 10"
set ylabel "Пакеты" font "Helvetica, 10"

# построение графика, используя значения
# 1-го и 5-го столбцов файла qm.out
plot "qm.out" using ($1):($5) with lines lt rgb "green" title "Размер очереди (в пакетах)", \
    "qm.out" using ($1):($5) smooth csplines lt rgb "red" title " Приближение сплайном ", \
    "qm.out" using ($1):($5) smooth bezier lt rgb "blue" title " Приближение Безье |"
```

Выполнение лабораторной работы

Сделаем файл исполняемым. После компиляции проекта запустим скрипт из файла `graph_plot` (см. рисунок [-@fig:005]), который создаст изображение `qm.png` с результатами моделирования (см. рисунок [-@fig:006]).

```
openmodelica@openmodelica-VirtualBox:~/work$ chmod +x graph_plot
openmodelica@openmodelica-VirtualBox:~/work$ ./graph_plot
```



Выводы

В процессе выполнения данной лабораторной работы я провела моделирование системы массового обслуживания (СМО).